

Elrond

A Highly Scalable Public Blockchain via Adaptive State Sharding and Secure Proof of Stake

[Technical whitepaper — release 2 — revision 1]

Updated chapters: [5 - 13]

June 19, 2019 - The Elrond Team

<https://www.elrond.com/>

Abstract—The advent of secure public blockchains through Bitcoin and later Ethereum, has brought forth a notable degree of interest and capital influx, providing the premise for a global wave of permissionless innovation. Despite lofty promises, creating a decentralized, secure and scalable public blockchain has proved to be a strenuous task.

This paper proposes Elrond, a novel architecture which goes beyond state of the art by introducing a genuine state sharding scheme for practical scalability, eliminating energy and computational waste while ensuring distributed fairness through a Secure Proof of Stake (SPoS) consensus. Having a strong focus on security, Elrond’s network is built to ensure resistance to known security problems like Sybil attack, Nothing at Stake attack and others. In an ecosystem that strives for interconnectivity, our solution for smart contracts offers an EVM compliant engine to ensure interoperability by design.

Preliminary simulations and testnet results reflect that Elrond exceeds Visa’s average throughput and achieves an improvement beyond three orders of magnitude or 1000x compared to the existing viable approaches, while drastically reducing the costs of bootstrapping and storage to ensure long term sustainability.

I Introduction

1 General aspects

Cryptocurrency and smart contract platforms such as Bitcoin and Ethereum have sparked considerable interest and have become promising solutions for electronic payments, decentralized applications and potential digital stores of value. However, when compared to their centralized counterparts in key metrics [1], the current state of affairs suggests that present public blockchain iterations exhibit severe limitations, particularly with respect to scalability, hindering their mainstream adoption and delaying public use. In fact, it has proved extremely challenging to deal with the current engineering boundaries imposed by the trade-offs in the blockchain trilemma paradigm [2]. Several solutions have been proposed, but few of them have shown significant and viable results. Thus, in order to solve the scalability problem, a complete rethinking of public blockchain infrastructures was required.

2 Defining the challenges

Several challenges must be addressed properly in the process of creating an innovative public blockchain solution designed to scale:

- **Full decentralization** - Eliminating the need for any trusted third party, hence removing any single point of failure;
- **Robust security** - Allowing secure transactions and preventing any attacks based on known attack vectors;
- **High scalability** - Enabling the network to achieve a performance at least equal to the centralized counterpart, as measured in TPS;
- **Efficiency** - Performing all network services with minimal energy and computational requirements;
- **Bootstrapping and storage enhancement** - Ensuring a competitive cost for data storage and synchronization;
- **Cross-chain interoperability** - Enforced by design, permitting unlimited communication with external services.

Starting from the above challenges, we’ve created Elrond as a complete rethinking of public blockchain infrastructure, specifically designed to be secure, efficient, scalable and interoperable. Elrond’s main contribution rests on two cornerstone building blocks:

- 1) **A genuine State Sharding approach:** effectively partitioning the blockchain and account state into multiple shards, handled in parallel by different participating validators;
- 2) **Secure Proof of Stake consensus mechanism:** an improved variation of Proof of Stake (PoS) that ensures long term security and distributed fairness, while eliminating the need for energy intensive PoW algorithms.

3 Adaptive State Sharding

Elrond proposes a dynamically adaptive sharding mechanism that enables shard computation and reorganizing based on necessity and the number of active network nodes. The reassignment of nodes in the shards at the beginning of each epoch is progressive and nondeterministic, inducing no temporary liveness penalties. Adaptive state sharding comes with additional challenges compared to the static model. One of the key-points resides in how shard-splitting and shard-merging is done to prevent overall latency penalties introduced by the synchronization/communication needs when the shard number changes. Latency, in this case, is the communication overhead required by nodes, in order to retrieve the new state, once their shard address space assignment has been modified.

Elrond proposes a solution for this problem below, but first some notions have to be defined: users and nodes. Users are external actors and can be identified by a unique account address; nodes are computers/devices in the Elrond network that run our protocol. Notions like users, nodes, addresses will be further described in chapter II.1 - Entities.

Elrond solves this challenge by:

- 1) Dividing the account address space in shards, using a binary tree which can be built with the sole requirement of knowing the exact number of shards in a certain epoch. Using this method, the accumulated latency is reduced and the network liveness is improved in two ways. First, thanks to the designed model, the dividing of the account address space is predetermined by hierarchy. Hence, there is no split overhead, meaning that one shard breaks into two shards, each of them keeping only one half of the previous address space in addition to the associated state. Second, the latency is reduced through the state redundancy mechanism, as the merge is prepared by retaining the state in the sibling nodes.
- 2) Introducing a technique of balancing the nodes in each shard, to achieve overall architecture equilibrium. This technique ensures a balanced workload and reward for each node in the network.
- 3) Designing a built-in mechanism for automatic transaction routing in the corresponding shards, considerably reduces latency as a result. The routing algorithm is described in chapter IV.4 - Elrond sharding approach.
- 4) In order to achieve considerable improvements with respect to bootstrapping and storage, Elrond makes use of a shard pruning mechanism. This ensures sustainability of our architecture even with a throughput of tens of thousands of transactions per second (TPS).

4 Secure Proof of Stake (SPoS)

We introduce a Secure Proof of Stake consensus mechanism, that expands on Algorand's [3] idea of a random selection mechanism, differentiating itself through the following aspects:

- 1) Elrond introduces an improvement which reduces the latency allowing each node in the shard to determine the members of the consensus group (block proposer and validators) at the beginning of a round. This is possible because the randomization factor r is stored in every block and is created by the block proposer using a BLS signature [4] on the previous r .
- 2) The block proposer is the validator in the consensus group who's hash of the public key and randomization factor is the smallest. In contrast to Algorand's [3] approach, where the random committee selection can take up to 12 seconds, in Elrond the time necessary for random selection of the consensus group is considerably reduced (estimated under 100 ms) excluding network latency. Indeed, there is no communication requirement for this random selection process, which enables Elrond to have a newly and randomly selected group that succeeds in committing a new block to the ledger in

each round. The tradeoff for this enhancement relies on the premise that an adversary cannot adapt faster than the round's time frame and can choose not to propose the block. A further improvement on the security of the randomness source, would be the use of verifiable delay functions (VDFs) in order to prevent any tampering possibilities of the randomness source until it is too late. Currently, the research in VDFs is still ongoing - there only a few working (and poorly tested) VDF implementations.

- 3) In addition to the stake factor generally used in PoS architectures as a sole decision input, Elrond refines its consensus mechanism by adding an additional weight factor called rating. The node's probability to be selected in the consensus group takes into consideration both stake and rating. The rating of a block proposer is recalculated at the end of each epoch, except in cases where slashing should occur, when the actual rating decrease is done instantly, adding another layer of security by promoting meritocracy.
- 4) A modified BLS multisignature scheme [5] with 2 communication rounds is used by the consensus group for block signing
- 5) Elrond considers formal verification for the critical protocol implementations (e.g. SPoS consensus mechanism) in order to validate the correctness of our algorithms.

II Architecture Overview

1 Entities

There are two main entities in Elrond: users and nodes. Users, each holding a (finite) number of public / private (Pk/sk) key pairs (e.g. in one or multiple wallet apps), use the Elrond network to deploy signed transactions for value transfers or smart contracts' execution. They can be identified by one of their account addresses (derived from the public key). The nodes are represented by the devices that form the Elrond network and can be passive or actively engaged in processing tasks. Eligible validators are active participants in Elrond's network. Specifically, they are responsible for running consensus, adding blocks, maintaining the state and being rewarded for their contribution. Each eligible validator can be uniquely identified by a public key constructed through a derivation of the address that staked the necessary amount and

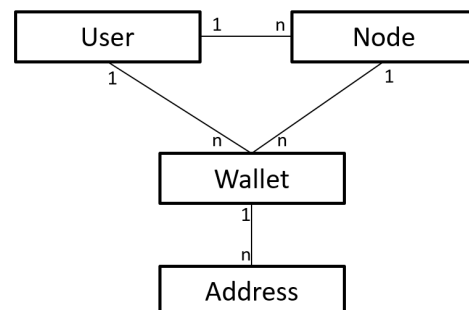


Fig. 1: Relations between Elrond entities

the node id. Relations between entities in the Elrond protocol are shown in Fig. 1.

Furthermore, the network is divided into smaller units called shards. An eligible validator is assigned to a shard based on an algorithm that keeps the nodes evenly distributed across shards, depending on the tree level. Each shard contains a randomly selected consensus group. Any block proposer is responsible to aggregate transactions into a new block. The validators are responsible to either reject, or approve the proposed block, thereby validating it and committing it to the blockchain.

2 Intrinsic token

Elrond grants access to the usage of its network through intrinsic utility tokens called *Elronds*, in short *ERDs*. All costs for processing transactions, running smart contracts and rewards for various contributions to the network will be paid in *ERDs*. References to fees, payments or balances are assumed to be in *ERDs*.

3 Threat model

Elrond assumes a byzantine adversarial model, where at least $\frac{2}{3}n+1$ of the eligible nodes in a shard are honest. The protocol permits the existence of adversaries that have stake or good rating, delay or send conflicting messages, compromise other nodes, have bugs or collude among themselves, but as long as $\frac{2}{3}n+1$ of the eligible validators in a shard are honest/not compromised, the protocol can achieve consensus.

The protocol assumes highly adaptive adversaries, which however cannot adapt faster than a round's timeframe. The computational power of an adversary is bounded, therefore the cryptographic assumptions granted by the security level of the chosen primitives hold firmly within the complexity class of problems solvable by a Turing machine in polynomial time.

The network of honest nodes is assumed to form a well connected graph and the propagation of their messages is done in a bounded time Δ .

Attack vectors' prevention

- 1) **Sybil attacks:** mitigated through the stake locking when joining the network. This way the generation of new identities has a cost equal to the minimum stake;
- 2) **Nothing at stake:** removed through the need of multiple signatures, not just from proposer, and the stake slashing. The reward per block compared to the stake locked will discourage such behavior;
- 3) **Long range attacks:** mitigated by our pruning mechanism, the use of a randomly selected consensus group every round (and not just a single proposer) and stake locking. On top of all these, our pBFT consensus algorithm ensures finality;
- 4) **DDoS attacks:** the consensus group is randomly sampled every round (few seconds); the small time frame making DDoS almost impossible.

Other attack vectors we have taken into consideration are: shard takeover attack, transaction censorship, double spend, bribery attacks, etc.

4 Chronology

In Elrond's network, the timeline is split into epochs and rounds. The epochs have a fixed duration, set to one day (can be modified as the architecture evolves), at the end of which the shards reorganization and pruning is triggered. The epochs are further divided into rounds, lasting for a fixed timeframe. A new consensus group is randomly selected per shard in each round, that can commit a maximum of one block in the shard's ledger.

New validators can join the network by locking their stake, as presented in chapter V.2 - Secure Proof of Stake. They are added to the unassigned node pool in the current epoch e , are assigned to the waiting list of a shard at the beginning of epoch $e + 1$, but can only become eligible validators to participate in consensus and get rewarded in the next epoch $e + 2$.

The timeline aspects are further detailed in section IX.1.

III Related Work

Elrond was designed upon and inspired by the ideas from Ethereum [6], Omniledger [7], Zilliqa [8], Algorand [3] and ChainSpace [9]. Our architecture goes beyond state of the art and can be seen as an augmentation of the existing models, improving the performance while focusing to achieve a better nash equilibrium state between security, scalability and decentralization.

1 Ethereum

Much of Ethereum's [6] success can be attributed to the introduction of its decentralized applications layer through EVM [10], Solidity [11] and Web3j [12]. While Dapps have been one of the core features of ethereum, scalability has proved a pressing limitation. Considerable research has been put into solving this problem, however results have been negligible up to this point. Still, few promising improvements are being proposed: Casper [13] prepares an update that will replace the current Proof of Work (PoW) consensus with a Proof of Stake (PoS), while Plasma based side-chains and sharding are expected to become available in the near future, alleviating Ethereum's scalability problem at least partially [14].

Compared to Ethereum, Elrond eliminates both energy and computational waste from PoW algorithms by implementing a SPoS consensus while using transaction processing parallelism through sharding.

2 Omniledger

Omniledger [7] proposes a novel scale-out distributed ledger that preserves long term security under permission-less operation. It ensures security and correctness by using a bias-resistant public-randomness protocol for choosing large, statistically representative shards that process transactions. To commit transactions atomically across shards, Omniledger introduces Atomix, an efficient cross-shard commit protocol. The concept is a two-phase client-driven "lock/unlock" protocol that ensures that nodes can either fully commit a transaction across shards, or obtain "rejection proofs" to abort and unlock

the state affected by partially completed transactions. Omniledger also optimizes performance via parallel intra-shard transaction processing, ledger pruning via collectively-signed state blocks, and low-latency "trust-but-verify" validation for low-value transactions. The consensus used in Omniledger is a BFT variation, named ByzCoinX, that increases performance and robustness against DoS attacks.

Compared to Omniledger, Elrond has an adaptive approach on state sharding, a faster random selection of the consensus group and an improved security by replacing the validators' set after every round (a few seconds) not after every epoch (1 day).

3 Zilliqa

Zilliqa [8] is the first transaction-sharding architecture that allows the mining network to process transactions in parallel and reach a high throughput by dividing the mining network into shards. Specifically, its design allows a higher transaction rate as more nodes are joining the network. The key is to ensure that shards process different transactions, with no overlaps and therefore no double-spending. Zilliqa uses pBFT [15] for consensus and PoW to establish identities and prevent Sybil attacks.

Compared to Zilliqa, Elrond pushes the limits of sharding by using not only transaction sharding but also state sharding. Elrond completely eliminates the PoW mechanism and uses SPoS for consensus. Both architectures are building their own smart contract engine, but Elrond aims not only for EVM compliance, so that SC written for Ethereum will run seamlessly on our VM, but also aims to achieve interoperability between blockchains.

4 Algorand

Algorand [3] proposes a public ledger that keeps the convenience and efficiency of centralized systems, without the inefficiencies and weaknesses of current decentralized implementations. The leader and the set of verifiers are randomly chosen, based on their signature applied to the last block's quantity value. The selections are immune to manipulations and unpredictable until the last moment. The consensus relies on a novel message-passing Byzantine Agreement that enables the community and the protocol to evolve without hard forks.

Compared to Algorand, Elrond doesn't have a single blockchain, instead it increases transaction's throughput using sharding. Elrond also improves on Algorand's idea of random selection by reducing the selection time of the consensus group from over 12 seconds to less than a second, but assumes that the adversaries cannot adapt within a round.

5 Chainspace

Chainspace [9] is a distributed ledger platform for high integrity and transparent processing of transactions. It uses language agnostic and privacy-friendly smart contracts for extensibility. The sharded architecture allows a linearly scalable transaction processing throughput using S-BAC, a novel distributed atomic commit protocol that guarantees consistency

and offers high auditability. Privacy features are implemented through modern zero knowledge techniques, while the consensus is ensured by BFT.

Compared to Chainspace, where the TPS decreases with each node added in a shard, Elrond's approach is not influenced by the number of nodes in a shard, because the consensus group has a fixed size. A strong point for Chainspace is the approach for language agnostic smart contracts, while Elrond focuses on building an abstraction layer for EVM compliance. Both projects use different approaches for state sharding to enhance performance. However, Elrond goes a step further by anticipating the blockchain size problem in high throughput architectures and uses an efficient pruning mechanism. Moreover, Elrond exhibits a higher resistance to sudden changes in node population and malicious shard takeover by introducing shard redundancy, a new feature for sharded blockchains.

IV Scalability via Adaptive State Sharding

1 Why sharding

Sharding was first used in databases and is a method for distributing data across multiple machines. This scaling technique can be used in blockchains to partition states and transaction processing, so that each node would process only a fraction of all transactions in parallel with other nodes. As long as there is a sufficient number of nodes verifying each transaction so that the system maintains high reliability and security, then splitting a blockchain into shards will allow it to process many transactions in parallel, and thus greatly improving transaction throughput and efficiency. Sharding promises to increase the throughput as the validator network expands, a property that is referred to as horizontal scaling.

2 Sharding types

A comprehensive and thorough introduction [16] emphasizes the three main types of sharding: network sharding, transaction sharding and state sharding. Network sharding handles the way the nodes are grouped into shards and can be used to optimize communication, as message propagation inside a shard can be done much faster than propagation to the entire network. This is the first challenge in every sharding approach and the mechanism that maps nodes to shards has to take into consideration the possible attacks from an attacker that gains control over a specific shard. Transaction sharding handles the way the transactions are mapped to the shards where they will be processed. In an account-based system, the transactions could be assigned to shards based on the sender's address. State sharding is the most challenging approach. In contrast to the previously described sharding mechanisms, where all nodes store the entire state, in state-sharded blockchains, each shard maintains only a portion of the state. Every transaction handling accounts that are in different shards, would need to exchange messages and update states in different shards. In order to increase resiliency to malicious attacks, the nodes in the shards have to be reshuffled from time to time. However, moving nodes between shards

introduces synchronization overheads, that is, the time taken for the newly added nodes to download the latest state. Thus, it is imperative that only a subset of all nodes should be redistributed during each epoch, to prevent down times during the synchronization process.

3 Sharding directions

Some sharding proposals attempt to only shard transactions [8] or only shard state [17], which increases transaction's throughput, either by forcing every node to store lots of state data or to be a supercomputer [2]. Still, more recently, at least one claim has been made about successfully performing both transaction and state sharding, without compromising on storage or processing power [13].

But sharding introduces some new challenges like: single-shard takeover attack, cross-shard communication, data availability and the need of an abstraction layer that hides the shards. However, conditional on the fact that the above problems are addressed correctly, state sharding brings considerable overall improvements: transaction throughput will increase significantly due to parallel transaction processing and transaction fees will be considerably reduced. Two main criterias widely considered to be obstacles transforming into advantages and incentives for mainstream adoption of the blockchain technology.

4 Elrond sharding approach

While dealing with the complexity of combining network, transaction and state sharding, Elrond's approach was designed with the following goals in mind:

- 1) **Scalability without affecting availability:** Increasing or decreasing the number of shards should affect a negligibly small vicinity of nodes without causing down-times, or minimizing them while updating states;
- 2) **Dispatching and instant traceability:** Finding out the destination shard of a transaction should be deterministic, trivial to calculate, eliminating the need for communication rounds;
- 3) **Efficiency and adaptability:** The shards should be as balanced as possible at any given time.

Method Description

To calculate an optimum number of shards N_{sh} in epoch e_{i+1} ($N_{sh,i+1}$), we have defined one threshold coefficient for the number of transactions in a block, θ_{TX} . Variable $optN$ represents the optimal number of nodes in a shard, ϵ_{sh} is a positive number and represents the number of nodes a shard can vary by. $totalN_i$ is the total number of nodes (eligible validators, nodes in the waiting lists and newly added nodes in the node pool) on all shards in epoch e_i , while $N_{TXB,i}$ is the average number of transactions in a block on all shards in epoch e_i . $N_{sh,0}$ will be considered as 1. The total number of shards $N_{sh,i+1}$ will change if the number of nodes $totalN_i$ in the network changes and if the blockchain utilization needs it: if the number of nodes increases above a threshold $nSplit$ from one epoch to another and the average number of transactions per block is greater than the threshold

number of transactions per block $N_{TXB,i} > \theta_{TX}$ or if the number of nodes decreases below a threshold $nMerge$ as shown in function *ComputeShardsN*.

```

1: function COMPUTESHARDSN( $totalN_{i+1}, N_{sh,i}$ )
2:    $nSplit \leftarrow (N_{sh,i} + 1) * (optN + \epsilon_{sh})$ 
3:    $nMerge \leftarrow (N_{sh,i} - 1) * a$ 
4:    $N_{sh,i+1} \leftarrow N_{sh,i}$ 
5:   if ( $totalN_{i+1} > nSplit$  and  $N_{TXB,i} > \theta_{TX}$ ) then
6:      $N_{sh,i+1} \leftarrow totalN_{i+1} / (optN + \epsilon_{sh})$ 
7:   else if  $totalN_{i+1} < nMerge$  then
8:      $N_{sh,i+1} \leftarrow totalN_{i+1} / (optN)$ 
9:   return  $N_{sh,i+1}$ 

```

From one epoch to another, there is a probability that the number of active nodes changes. If this aspect influences the number of shards, anyone can calculate the two masks m_1 and m_2 , used in transaction dispatching.

```

1: function COMPUTEM1ANDM2( $N_{sh}$ )
2:    $n \leftarrow \text{math.ceil}(\log_2 N_{sh})$ 
3:    $m_1 \leftarrow (1 << n) - 1$ 
4:    $m_2 \leftarrow (1 << (n - 1)) - 1$ 
5:   return  $m_1, m_2$ 

```

As the main goal is to increase the throughput beyond thousands of transactions per second and to diminish the cross-shard communication, Elrond proposes a dispatching mechanism which determines automatically the shards involved in the current transaction and routes the transaction accordingly. The dispatcher will take into consideration the account address (*addr*) of the transaction sender/receiver. The result is the number of the shard (*shard*) the transaction will be dispatched to.

```

1: function COMPUTESHARD( $N_{sh}, addr, m_1, m_2$ )
2:    $shard \leftarrow (addr \text{ and } m_1)$ 
3:   if  $shard > N_{sh}$  then
4:      $shard \leftarrow (addr \text{ and } m_2)$ 
5:   return  $shard$ 

```

The entire sharding scheme is based on a binary tree structure that distributes the account addresses, favors the scalability and deals with the state transitions. A representation of the tree can be seen in Fig. 2.

The presented tree structure is merely a logical representation of the account address space used for a deterministic mapping; e.g. shard allocation, sibling computation etc. The leaves of the binary tree represent the shards with their ID number. Starting from root (node/shard 0), if there is only one shard/leaf (a), all account addresses are mapped to this one and all transactions will be executed here. Further on, if the formula for N_{sh} dictates the necessity of 2 shards (b), the address space will be split in equal parts, according to the last bits in the address.

Sometimes, the tree can also become unbalanced (c) if N_{sh} is not a power of 2. This case only affects the leaves on the

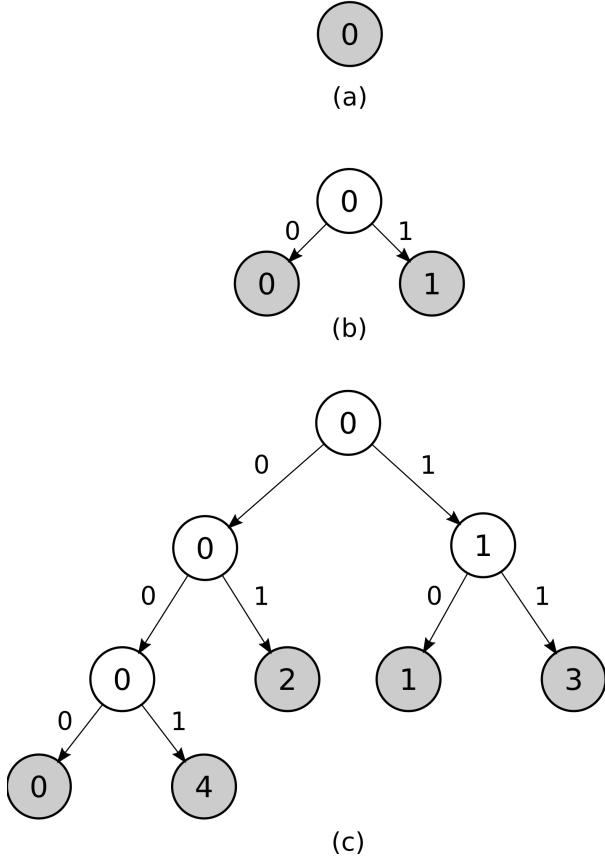


Fig. 2: Example of a sharding tree structure

last level. The structure will become balanced again when the number of shards reaches a power of 2.

The unbalancing of the binary tree causes the shards located in the lowest level to have half the address space of nodes of a shard located one level higher, so it can be argued that the active nodes allocated to these shards will have a lower fee income - block rewards are not affected. However, this problem is solved by having a third of each shard nodes redistributed randomly each epoch (detailed in the Chronology section) and having a balanced distribution of nodes according to the tree level.

Looking at the tree, starting from any leaf and going through branches towards the root, the encoding from branches represents the last n bits of the account addresses that will have their associated originating transactions processed by that leaf/shard. Going the other way around, from root to leaf, the information is related to the evolution of the structure, sibling shards, the parent shard from where they split. Using this hierarchy, the shard that will split when N_{sh} increases or the shards that will merge when N_{sh} decreases can easily be calculated. The entire state sharding mechanism benefits from this structure by always keeping the address and the associated state within the same shard.

Knowing N_{sh} , any node can follow the redistribution process without the need of communication. The allocation of ID's for the new shards is incremental and reducing the number of shards involves that the higher numbered shards

will be removed. For example, when going from N_{sh} to $N_{sh}-1$, two shards will be merged, the shard to be removed is the highest numbered shard ($sh_{merge}=N_{sh}-1$). Finding the shard number that sh_{merge} will be merged with is trivial. According to the tree structure, the resulting shard has the sibling's number:

```

1: function COMPUTESIBLING( $sh_{merge}, n$ )
2:    $sibling \leftarrow (sh_{merge} \mathbf{xor} (1 << (n - 1)))$ 
3:   return  $sibling$ 

```

For shard redundancy, traceability of the state transitions and fast scaling, it is important to determine the *sibling* and *parent* of a generic shard with number p :

```

1: function COMPUTEPARENTSIBLINGS( $n, p, N_{sh}$ )
2:    $mask_1 \leftarrow 1 << (n - 1)$ 
3:    $mask_2 \leftarrow 1 << (n - 2)$ 
4:    $sibling \leftarrow (p \mathbf{xor} mask_1)$ 
5:    $parent \leftarrow \min(p, sibling)$ 
6:   if  $sibling \geq N_{sh}$  then
7:      $sibling \leftarrow (p \mathbf{xor} mask_2)$ 
8:      $sibling_2 \leftarrow (sibling \mathbf{xor} mask_1)$ 
9:      $parent \leftarrow \min(p, sibling)$ 
10:    if  $sibling_2 \geq N_{sh}$  then  $\triangleright$  sibling is a shard
11:      return  $parent, sibling, NULL$ 
12:    else
13:       $\triangleright$  sibling is a subtree with
14:       $\triangleright$  shards ( $sibling, sibling_2$ )
15:      return  $parent, sibling, sibling_2$ 
16:  else  $\triangleright$  sibling is a shard
17:    return  $parent, sibling, NULL$ 

```

Shard redundancy

On blockchain, state sharding is susceptible to shard failure when there is an insufficient number of online nodes in a shard or the distribution is localized geographically. In the unlikely case when one shard fails (either the shard cannot be contacted - all nodes are offline, or consensus cannot be reached - more than $\frac{1}{3}$ of nodes are not responding), there is a high risk that the entire architecture relies only on super-full nodes [2], which fully download every block of every shard, fully verifying everything. As displayed in Fig. 3, our protocol has a protection mechanism that introduces a tradeoff in the state holding structure by enforcing the shards from the last tree level to also hold the state from their siblings. This mechanism reduces the communication and eliminates the bootstrapping when sibling shards are merging since they already have the data.

Context switching

To preserve security in sharded public blockchains, context switching becomes crucial [7]. This refers to the reallocation of the active nodes between shards on a fixed time interval by some random criteria. In Elrond's approach, the context switching represents a security improvement, but also increases the complexity required to maintain consistency between multiple states. The state transition has the biggest

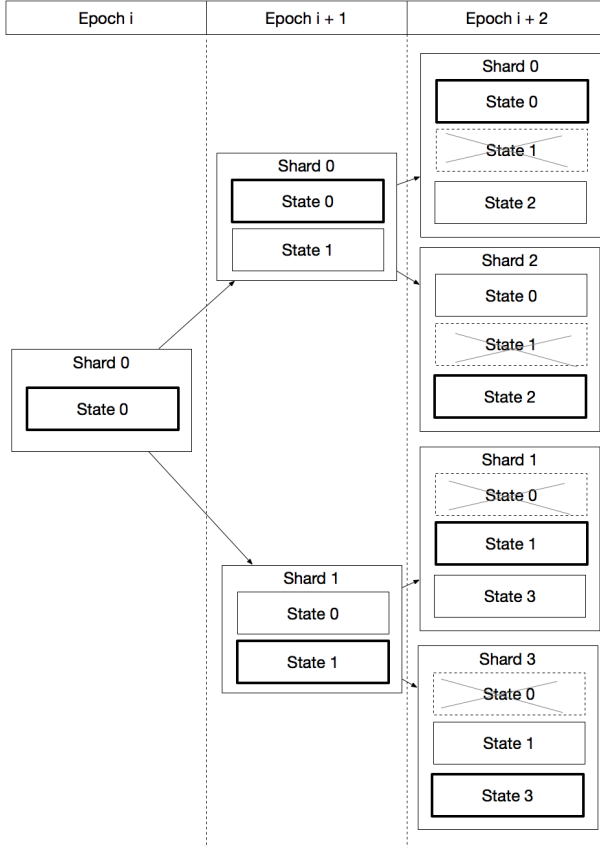


Fig. 3: Shard redundancy across epochs

footprint on performance since the movement of active nodes requires to resync the state, blockchain and transactions alongside the eligible nodes in the new shard. At the start of each epoch, in order to maintain liveness, only less than $\frac{1}{3}$ of these nodes will be uniformly re-distributed across shards. This mechanism is highly effective against forming malicious groups.

5 Notarization (Meta) chain

All network and global data operations (node joining the network, node leaving the network, eligible validator lists computation, nodes assignment to the shard's waiting lists, consensus agreement on a block in a specific shard challenges for invalid blocks will be notarized in the metachain. The metachain consensus is run by a different shard that communicates with all other shards and facilitates cross-shard operations. Every round of every epoch, the metachain receives block headers from the other shards and, if necessary, proofs for the challenges of the invalid blocks. This information will be aggregated into blocks on the metachain on which consensus has to be run. Once the blocks are validated in the consensus group, shards can request information about blocks, miniblocks (see chapter VII), eligible validators, nodes in waiting lists etc., in order to securely process cross-shard transactions. Further details about the cross-shard transaction execution, communication between shards and metachain will

be presented in Chapter VII Cross-shard transaction processing.

V Consensus via Secure Proof of Stake

1 Consensus Analysis

The first blockchain consensus algorithm based on Proof of Work (PoW), is used in Bitcoin, Ethereum and other blockchain platforms. In Proof of Work each node is required to solve a mathematical puzzle (hard to calculate but easy to verify). And the first node that finishes the puzzle will collect the reward [18]. Proof of Work mechanisms successfully prevent double-spending, DDoS and Sybil attacks at the cost of high energy consumption.

Proof of Stake (PoS) is a novel and more efficient consensus mechanism proposed as an alternative to the intensive energy and computational use in Proof of Work consensus mechanisms. PoS can be found in many new architectures like Cardano [19] and Algorand [3] or can be used in next version of Ethereum. In PoS the node that proposes the next block is selected by a combination of stake (wealth), randomness and/or age. It mitigates the PoW energy problem but also puts two important issues on the table: the Nothing at Stake attack and a higher centralization risk.

Proof of Meme as envisioned in Constellation [20], is an algorithm based on the node's historical participation on the network. Its behaviour is stored in a matrix of weights in the blockchain and supports changes over time. Also, it allows new nodes to gain trust by building up reputation. The main drawback regarding Sybil attacks is alleviated through the NetFlow algorithm.

Delegated Proof of Stake (DPoS) found in Bitshares [21], Steemit [22] and EOS [23] is a hybrid between Proof of Authority and Proof of Stake in which the few nodes responsible for deploying new blocks are elected by stakeholders. Although it has a high throughput, the model is susceptible to human related social problems such as bribing and corruption. Also, a small number of delegates makes the system prone to DDoS attacks and centralization.

2 Secure Proof of Stake (SPoS)

Elrond's approach to consensus is made by combining random validators' selection, eligibility through stake and rating, with an optimal dimension for the consensus group. The algorithm is described in the steps below:

- 1) Each node n_i is defined as a tuple of public key (Pk), rating (default is 0) and the locked stake. If n_i wishes to participate in the consensus, it has to first register through a smart contract, by sending a transaction that contains an amount equal to the minimum required stake and other information (Pk_s , a public key derived from Pk and $nodeid$ that will be used for the signing process in order not to use a real wallet address).
- 2) The node n_i joins the node pool and waits for the shard assignment at the end of the current epoch e . The shard assignment mechanism creates a new set of nodes containing all the nodes that joined in epoch e and all

the nodes that need to be reshuffled (less than $\frac{1}{3}$ of every shard). All nodes in this set will be reassigned to the waiting lists of shards. W_j represents j 's shard waiting list and N_{sh} represents the number of shards. A node also has a secret key sk that by nature is not to be made public.

$$n_i = (Pk_i, rating_i, stake_i)$$

$$n_i \in W_j, 0 \leq j < N_{sh}$$

- 3) At the end of the epoch in which it has joined, the node will be moved to the list of eligible nodes (E_j) of a shard j , where e is the current epoch.

$$n_i \in W_{j,e-1} \rightarrow n_i \notin W_{j,e}, n_i \in E_{j,e}$$

- 4) Each node from the list E_j can be selected as part of an optimally dimensioned consensus group (in terms of security and communication), by a deterministic function, based on the randomness source added to the previous block, the round r and a set of variation parameters. The random number, known to all shard nodes through gossip, cannot be predicted before the block is actually signed by the previous consensus group. This property makes it a good source of randomness and prevents highly adaptive malicious attacks. We define a selection function to return the set of chosen nodes (consensus group) N_{chosen} with the first being the block proposer, that takes following parameters: E , r and sig_{r-1} - the previous block signature.

$$N_{chosen} = f(E, r, sig_{r-1}), \text{ where } N_{chosen} \subset E$$

- 5) The block will be created by the block proposer and the validators will co-sign it based on a modified practical Byzantine Fault Tolerance (pBFT).
- 6) If, for any reason, the block proposer did not create a block during its allocated time slot (malicious, offline, etc.), round r will be used together with the randomness source from the last block to select a new consensus group.

If the current block proposer acts in a malicious way, the rest of the group members apply a negative feedback to change its rating, decreasing or even cancelling out the chances that this particular node will be selected again. The feedback function for the block proposer (n_i) in round number r , with parameter $ratingModifier \in \mathbb{Z}$ is computed as:

$$feedbackfunction = ff(n_i, ratingModifier, r)$$

When $ratingModifier < 0$, slashing occurs so the node n_i loses its stake.

The consensus protocol remains safe in the face of DDoS attacks by having a high number of possible validators from the list E (hundreds of nodes) and no way to predict the order of the validators before they are selected.

To reduce the communication overhead that comes with an increased number of shards, a consensus will be run on a composite block. This composite block is formed by:

- **Ledger block:** the block to be added into the shard's

ledger, having all intra shard transactions and cross shard transactions for which confirmation proof was received;

- **Multiple mini-blocks:** each of them holding cross shard transactions for a different shard;

The consensus will be run only once, on the composite block containing both intra- and cross-shard transactions. After consensus is reached, the block header of each shard is sent to the metachain for notarization.

VI Cryptographic Layer

1 Signature Analysis

Digital signatures are cryptographic primitives used to achieve information security by providing several properties like message authentication, data integrity and non-repudiation [24].

Most of the schemes used for existing blockchain platforms rely on the discrete logarithm (DL) problem: one-way exponentiation function $y \rightarrow \alpha^y \text{ mod } p$. It is scientifically proven that calculating the discrete logarithm with base is hard [25].

Elliptic curve cryptography (ECC) uses a cyclic group of points instead of a cyclic group of integers. The scheme reduces the computational effort, such that for key lengths of only 160 - 256 bits, ECC provides same security level that RSA, Elgamal, DSA and others provide for key lengths of 1024 - 3072 bits (see Table 1 [24]).

The reason why ECC provides a similar security level for much smaller parameter lengths is because existing attacks on elliptic curve groups are weaker than the existing integer DL attacks, the complexity of such algorithms require on average $\sqrt{p}$ steps to solve. This means that an elliptic curve using a prime p of 256 bit length provides on average a security of 2^{128} steps needed to break it [24].

Both Ethereum and Bitcoin use curve cryptography, with the ECDSA signing algorithm. The security of the algorithm is very dependent on the random number generator, because if the generator does not produce a different number on each query, the private key can be leaked [26].

Another digital signature scheme is EdDSA, a Schnorr variant based on twisted Edwards curves that support fast arithmetic [27]. In contrast to ECDSA, it is provably non-malleable, meaning that starting from a simple signature, it is impossible to find another set of parameters that defines the same point on the elliptic curve [28], [29]. Additionally, EdDSA doesn't need a random number generator because it

Algorithm Family	Crypto systems	Security level (bit)			
		80	128	192	256
Integer factorization	RSA	1024	3072	7680	15360
Discrete logarithm	DH, DSA, Elgamal	1024	3072	7680	15360
Elliptic curves	ECDH, ECDSA	160	256	384	512
Symmetric key	AES, 3DES	80	128	192	256

TABLE 1: Bit lengths of public-key algorithms for different security levels

uses a nonce, calculated as the hash of the private key and the message, so the attack vector of a broken random number generator that can reveal the private key is avoided.

Schnorr signature variants are gaining more attention [8], [30] due to a native multi-signature capability and being provably secure in the random oracle model [31]. A multi-signature scheme is a combination of a signing and verification algorithms, where multiple signers, each with their own private and public keys, can sign the same message, producing a single signature [32], [33]. This signature can then be checked by a verifier which has access to the message and the public keys of the signers. A sub-optimal method would be to have each node calculate his own signature and then concatenate all results in a single string. However, such an approach is unfeasible as the generated string size grows with the number of signers. A practical solution would be to aggregate the output into a single fixed size signature, independent of the number of participants. There have been multiple proposals of such schemes, most of them are susceptible to rogue-key (cancellation) attacks. One solution for this problem would be to introduce a step where each signer needs to prove possession of the private key associated with its public key [34].

Bellare and Neven [35] (BN) proposed a secure multi-signature scheme without a proof of possession, in the plain public key model, under the discrete logarithm assumption [31]. The participants commit first to their share R_i by propagating its hash to all other signers so they cannot calculate a function of it. Each signer computes a different challenge for their partial signature. However, this scheme sacrifices the public key aggregation. In this case, the verification of the aggregated signature, requires the public key from each signer.

A recent paper by Gregory Maxwell et al. [29] proposes another multi-signature scheme in the plain public key model [36], under the 'one more discrete logarithm' assumption (OMDL). This approach improves the previous scheme [35] by reducing the communication rounds from 3 to 2, reintroducing the key aggregation with a higher complexity cost.

BLS [4] is another interesting signature scheme, from the Weil pairing, which bases its security on the Computational Diffie-Hellman assumption on certain elliptic curves and generates short signatures. It has several useful properties like batch verification, signature aggregation, public key aggregation, making BLS a good candidate for threshold and multi-signature schemes.

Dan Boneh, Manu Drijvers and Gregory Neven recently proposed a BLS multi-signature scheme [5], using ideas from the previous work of [35], [30] to provide the scheme with defenses against rogue key attacks. The scheme supports efficient verification with only two pairings needed to verify a multi-signature and without any proof of knowledge of the secret key (works in the plain public key model). Another advantage is that the multi-signature can be created in only two communication rounds.

For traceability and security reasons, a consensus based on a reduced set of validators requires the public key from each signer. In this context, our analysis concludes that the most appropriate multi-signature scheme for block signing in Elrond

is BLS multi-signature [5], which is faster overall than the other options due to only two communication rounds.

2 Block signing in Elrond

For block signing, Elrond uses curve cryptography based on the BLS multi-signature scheme over the $bn256$ bilinear group, which implements the Optimal Ate pairing over a 256-bit Barreto Naehrig curve. The bilinear pairing is defined as:

$$e : g_0 \times g_1 \rightarrow g_t \quad (1)$$

where g_0 , g_1 and g_t are elliptic curves of prime order p defined by $bn256$, and e is a bilinear map (i.e. pairing function). Let G_0 and G_1 be generators for g_0 and g_1 . Also, let H_0 be a hashing function that produces points on the curve g_0 :

$$H_0 : \mathcal{M} \rightarrow g_0 \quad (2)$$

where $\mathcal{M}$ is the set of all possible binary messages of any length. The signing scheme used by Elrond employs a second hashing function as well, with parameters known by all signers:

$$H_1 : \mathcal{M} \rightarrow Z_p \quad (3)$$

Each signer i has its own private/public key pair (sk_i, Pk_i) , where sk_i is randomly chosen from Z_p . For each key pair, the property $Pk_i = sk_i \cdot G_1$ holds.

Let $L = Pk_1, Pk_2, \dots, Pk_n$ be the set of public keys of all possible signers during a specific round which, in the case of Elrond, is the set of public keys of all the nodes in the consensus group. Below, the two stages of block signing process is presented: signing and verification.

Practical signing - Round 1

The leader of the consensus group creates a block with transactions, then signs and broadcasts this block to the consensus group members.

Practical signing - Round 2

Each member of the consensus group (including the leader) who receives the block must validate it, and if found valid, it signs it with BLS and then sends the signature to the leader:

$$Sig_i = sk_i * H_0(m) \quad (4)$$

where Sig_i is a point on g_0 .

Practical signing - Round 3

The leader waits to receive the signatures for a specific timeframe. If it does not receive at least $\frac{2}{3} \cdot n + 1$ signatures in that timeframe, the consensus round is aborted. But if the leader does receive $\frac{2}{3} \cdot n + 1$ or more valid signatures, it uses them to generate the aggregated signature:

$$SigAgg = \sum_i H_1(Pk_i) \cdot Sig_i \cdot B[i] \quad (5)$$

where $SigAgg$ is a point on g_0 .

The leader then adds the aggregated signature to the block together with the selected signers bitmap B , where a 1 indicates that the corresponding signer in the list L had its signature added to the aggregated signature $SigAgg$.

Practical verification

Given the list of public keys L , the bitmap for the signers B , the aggregated signature $SigAgg$, and a message m (block), the verifier computes the aggregated public key:

$$PkAgg = \sum_i H_1(Pk_i) \cdot Pk_i \cdot B_i \quad (6)$$

The result, $PkAgg$, is a point on g_1 . The final verification is

$$e(G_1, SigAgg) == e(PkAgg, H_0(m)) \quad (7)$$

where e is the pairing function.

VII Cross-shard Execution

For an in depth example of how the cross-shard transactions are being executed and how the communication between shards and the metachain occurs, we are simplifying the entire process to just two shards and the metachain. Assuming that a user generates a transaction from his wallet, which has an address in shard 0 and wants to send ERDs to another user that has a wallet with an address in shard 1, the steps depicted in Fig. 4 are required for processing the cross-shard transaction.

As mentioned in chapter V - Consensus via Secure Proof of Stake, the blocks structure is represented by a block Header that contains information about the block (block nonce, round, proposer, validators timestamp etc), and a list of miniblocks for each shard that contain the actual transactions inside. Every miniblock contains all transactions that have either the sender in the current shard and the receiver in another shard or the sender in a different shard and the destination in the current shard. In our case, for a block in shard 0, there will normally be 3 miniblocks:

- miniblock 0: containing the intrashard transactions for shard 0

- miniblock 1: containing cross-shard transactions with the sender in shard 0 and destination in shard 1
- miniblock 2: containing cross-shard transactions with sender in shard 1 and destination in shard 0. These transactions were already processed in the sender shard 1 and will be finalized after the processing also in the current shard.

There is no limitation on the number of miniblocks with the same sender and receiver in one block. Meaning multiple miniblocks with the same sender and receiver can appear in the same block.

1 Processing

Currently the atomic unit of processing in cross-shard execution is a miniblock: either all the transactions of the miniblock are processed at once or none and the miniblock's execution will be retried in the next round.

Our cross-shard transaction strategy uses an asynchronous model. Validation and processing is done first in sender's shard and then in receivers' shard. Transactions are first dispatched in the sender's shard, as it can fully validate any transaction initiated from the account in this shard – mainly the current balance. Afterwards, in the receivers' shard, the nodes only need proof of execution offered by metachain, do signature verification and check for replay attack and finally update the balance for the receiver, adding the amount from the transaction.

Shard 0 processes both intra-shard transactions in miniblock 0 and a set of cross-shard transactions that have addresses from shard 1 as a receiver in miniblock 1. The block header and miniblocks are sent to the metachain. The metachain notarizes the block from shard 0, by creating a new metachain block (metablock) that contains the following information about each miniblock: sender shard ID, receiver shard ID, miniblock hash.

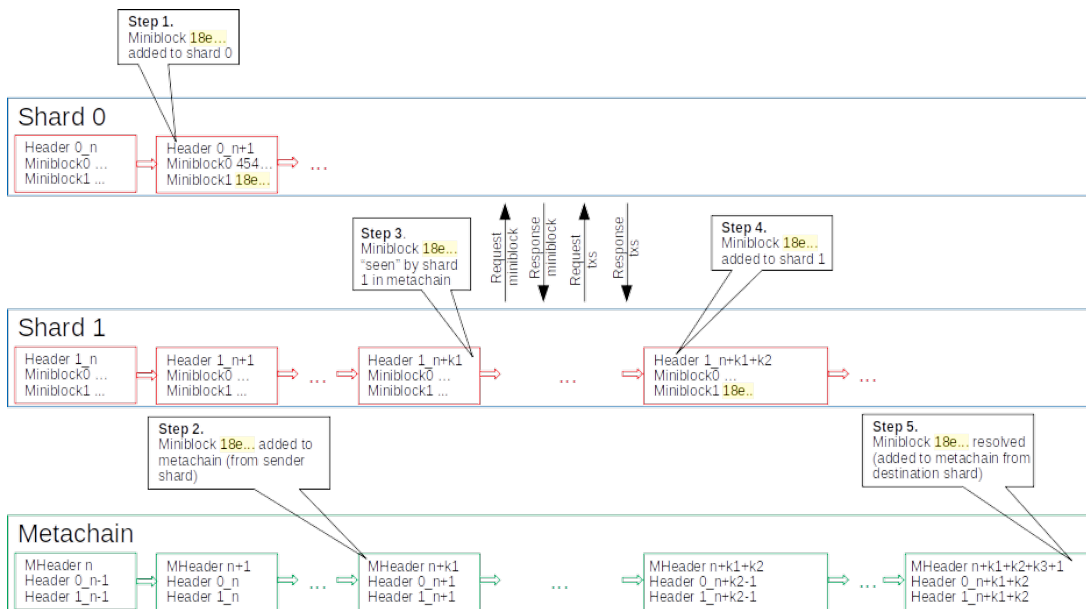


Fig. 4: Cross-shard transaction processing

Shard 1 fetches the hash of miniblock 1 from metablock, requests the miniblock from shard 0, parses the transaction list, requests missing transactions (if any), executes the same miniblock 1 in shard 1 and sends to the metachain resulting block. After notarization the cross transaction set can be considered finalized.

The next diagram shows the number of rounds required for a transaction to be finalized. The rounds are considered between the first inclusion in a miniblock until the last miniblock is notarised.

VIII Smart Contracts

The execution of smart contracts is a key element in all future blockchain architectures. Most of the existing solutions avoid to properly explain the transactions and data dependency. This context leads to the following two scenarios:

- 1) When there is no direct correlation between smart contract transactions, as displayed in Fig. 5, any architecture can use out of order scheduling. This means there are no additional constraints on the time and place (shard) where a smart contract is executed.
- 2) The second scenario refers to the parallelism induced by the transactions that involve correlated smart contracts [37]. This case, reflected in Fig. 6, adds additional pressure on the performance and considerably increases the complexity. Basically there must be a mechanism to ensure that contracts are executed in the right order and on the right place (shard). To cover this aspect, Elrond protocol proposes a solution that assigns and moves the smart contract to the same shard where their static dependencies reside. This way most, if not all SC calls will have dependencies in the same shard and no cross-shard locking/unlocking will be needed.

Elrond focuses on the implementation of the Elrond Virtual Machine, an EVM compliant engine. The EVM compliance

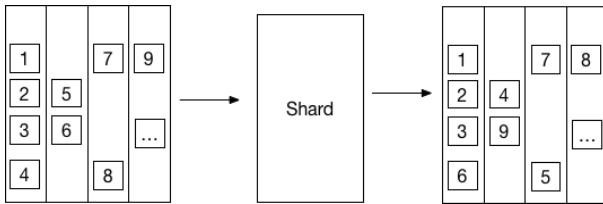


Fig. 5: Independent transaction processing under simple smart contracts that can be executed out of order

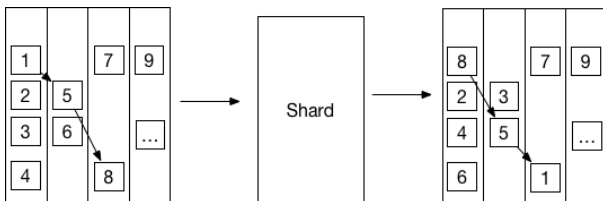


Fig. 6: Mechanism for correlated smart contracts that can be executed only sequentially

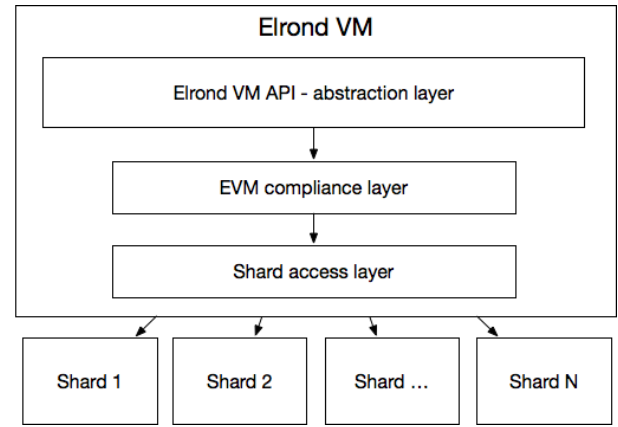


Fig. 7: Abstraction Layer for Smart Contracts

is extremely important for adoption purposes, due to the large number of smart contracts built on Ethereum's platform.

The Elrond Virtual Machine's implementation will hide the underlying architecture isolating the smart contract developers from system internals ensuring a proper abstraction layer, as displayed in Fig. 7.

In Elrond, cross chain interoperability can be implemented by using an adapter mechanism at the Virtual Machine level as proposed by Cosmos [38]. This approach requires specialized adapters and an external medium for communication between adapter SC for each chain that will interoperate with Elrond. The value exchange will be operated using some specialized smart contracts acting as asset custodians, capable of taking custody of adapted chain native tokens and issuing Elrond native tokens.

1 VM Infrastructure

Elrond builds its VM infrastructure on top of the K Framework, which is an executable semantic framework where programming languages, calculi, as well as type systems or formal analysis tools can be defined [39].

The greatest advantage of using the K framework is that with it, smart contract languages can be unambiguously defined, eliminating the potential for unspecified behavior and bugs that are hard to detect.

The K Framework is executable, in the sense that the semantic specifications of languages can be directly used as working interpreters for the languages in question. More specifically, one can either run programs against the specifications using the K Framework core implementation directly, or one can generate an interpreter in several programming languages. These are also referred to as "backends". For the sake of execution speed and ease of interoperability, Elrond uses its own custom-built K Framework backend.

2 Smart contract languages

One great advantage of the K Framework is that one can generate an interpreter for any language defined in K, without the need for additional programming. This also means that interpreters produced this way are "correct-by-construction".

There are several smart contract languages specified in the K Framework already, or with their specifications under development. Elrond Network will support three low-level languages: IELE VM, KEVM, and WASM.

- IELE VM is an intermediate-level language, in the style of LLVM, but adapted for the blockchain. It was built directly in K, no other specification or implementation of it exists outside of the K framework [40]. Its purpose is to be human readable, fast, and to overcome some limitations of EVM. Elrond uses a slightly altered version of IELE - most changes are related to account address management. Smart contract developers can program in IELE directly, but most will choose to code in Solidity and then use a Solidity to IELE compiler, as can be seen in Fig. 8.
- KEVM is a version of the Ethereum Virtual Machine (EVM), written in K [41]. Certain vulnerabilities of EVM are fixed in the K version, or the vulnerable features are left out entirely.
- Web Assembly (WASM) is a binary instruction format for a stack-based virtual machine, which can be used for running smart contracts. A WASM infrastructure enables developers to write smart contracts in C/C++, Rust, C#, and others.

Having a language specification and generating the interpreter is only half of the challenge. The other half is integrating the generated interpreter with the Elrond network. We have built a common VM interface, that enables us to plug in any VM into an Elrond node as shown in Fig. 9. Each VM then has an adapter that implements this interface. Each contract is saved as bytecode of the VM for which it was compiled and runs on its corresponding VM.

3 Support for formal modelling and verification

Because the smart contract languages are formally defined in K Framework, it is possible to perform formal verification of smart contracts written in these languages. To do this, it is necessary to also formally model their requirements, which can also be performed using the K Framework [42].

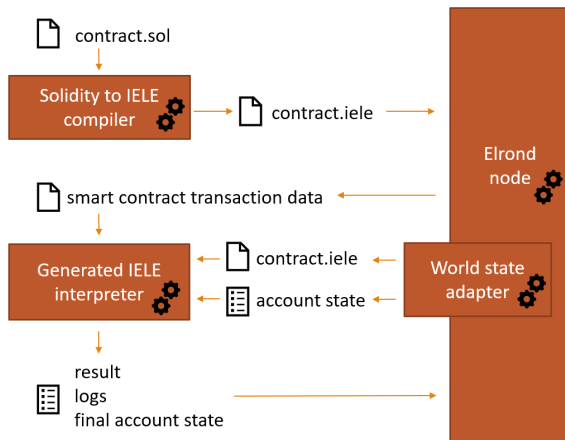


Fig. 8: Elrond VM execution

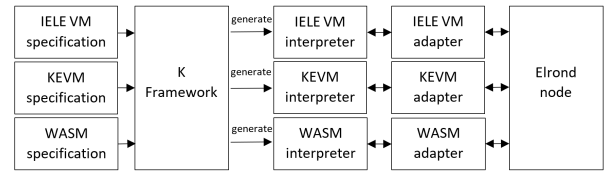


Fig. 9: Elrond VM components

4 Smart contracts on the sharded architecture

Smart contracts on sharded architectures are still in the early stages of research and development and pose serious challenges. Protocols like Atomix [7] or S-BAC [9] represent a starting point. Dynamic smart contract dependencies cannot be resolved by moving the SCs into the same shard, as at deployment time, not all the dependencies can be calculated.

Solution currently research in the space:

- 1) A locking mechanism that allows the atomic execution of smart contract from different shards, ensures that the involved SCs will be either all executed at the same time, or none at all. This requires multiple interaction messages and synchronization between consensus of different shards. [9]
- 2) Cross-shard contract yanking proposal for Ethereum 2.0 would move that smart contract code and data into the caller shard at the execution time. Atomic execution is not needed, but the locking mechanism is mandatory on the moved SC, which would block the execution of SC for other transactions. The locking mechanism is simpler, but it needs to transfer the whole internal state of the SC. [43]

Following Ethereum's model, Elrond has the following transaction types:

- 1) SC construction and deployment: transactions receiver address is empty and data field contains the smart contract code as byte array;
- 2) SC method invoking: transaction has a non empty receiver address and that address has an associated code;
- 3) Payment transactions: transaction has a non empty receiver and that address does not have code.

Elrond's approach to this problem is to use asynchronous cross-shard execution model in case of smart contracts. The user creates a smart contract execution transaction. If the smart contract is not in the current shard, the transaction is treated as a payment transaction, the value of the transaction is subtracted from the sender account and it is added to the block where the sender shard resides, into a miniblock with the destination shard where the receiver account is. The transaction is notarized by metachain, then processed by the destination shard. In the destination shard, the transaction is treated as SC method invoking, as the receiver address is a smart contract which exists in this shard. For the smart contract call a temporary account which shadows the sender account is created, with the balance from the transaction value and the smart contract is called. After the execution, the smart contract might return results which affects a number

of accounts from different shards. All the results, which affect in-shard accounts are executed in the same round. For those accounts which are not in the shard where the smart contract was executed, transactions called Smart Contract Results will be created, saving the smart contract execution output for each of these accounts. SCR miniblocks are created for each destination shard. These miniblocks are notarized the same way as cross-shard transactions by metachain, then processed by the respective shards, where the accounts resides. In case one smart contract calls dynamically another smart contract from another shard, this call is saved as an intermediate result and treated the same as for accounts.

The solution has multiple steps and the finalization of a cross-shard smart contract call will need at least 5 rounds, but it does not need locking and state movement across shards.

IX Bootstrapping and Storage

1 Timeline division

Proof of Stake systems tend to generally divide timeline into epochs and each epoch into smaller rounds [19]. The timeline and terminology may differ between architectures but most of them use a similar approach.

Epochs

In Elrond Protocol, each epoch has a fixed duration, initially set to 24 hours (might suffer updates after several testnet confirmation stages). During this timeframe, the configuration of the shards remains unchanged. The system adapts to scalability demands between epochs by modifying the number of shards. To prevent collusion, after an epoch, the configuration of each shard needs to change. While reshuffling all nodes between shards would provide the highest security level, it would affect the system's liveness by introducing additional latency due to bootstrapping. For this reason, at the end of each epoch, less than $\frac{1}{3}$ of the eligible validators, belonging to a shard will be redistributed non-deterministically and uniformly to the other shards' waiting lists.

Only prior to the start of a new epoch, the validator distribution to shards can be determined, without additional communication as displayed in Fig. 10.

The node shuffling process runs in multiple steps:

- 1) The new nodes registered in the current epoch e_i land in the unassigned node pool until the end of the current epoch;
- 2) Less than $\frac{1}{3}$ of the nodes in every shard are randomly selected to be reshuffled and are added to the assigned node pool;
- 3) The new number of shards $N_{sh,i+1}$ is computed based on the number of nodes in the network k_i and network usage;
- 4) Nodes previously in all shard's waiting lists, that are currently synchronized, are added to the eligible validator's lists;
- 5) The newly added nodes from the unassigned node pool are uniformly random distributed across all shards' waiting lists during epoch e_{i+1} ;

- 6) The reshuffled nodes from the assigned node pool are redistributed with higher ratios to shards' waiting lists that will need to split in the next epoch e_{i+2} .

Rounds

Each round has a fixed time duration of 5 seconds (might suffer updates after several testnet confirmation stages). During each round, a new block can be produced within every shard by a randomly selected set of block validators (including one block proposer). From one round to another the set is changed using the eligible nodes list, as detailed in the chapter IV.

As described before, the reconfiguration of shards within epochs and the arbitrary selection of validators within rounds discourages the creation of unfair coalitions, diminishes the possibility of DDoS and bribery attacks while maintaining decentralization and a high transactions throughput.

2 Pruning

A high throughput will lead to a distributed ledger that rapidly grows in size and increases bootstrapping cost (time+storage), as highlighted in section XI.1.

This cost can be addressed by using efficient pruning algorithms, that can summarize the blockchain's full state in a more condensed structure. The pruning mechanism is similar to the stable checkpoints in pBFT [15] and compresses the entire ledger state.

Elrond protocol makes use of an efficient pruning algorithm [7] detailed below. Let us consider that e is the current epoch and a is the current shard:

- 1) the shard nodes keep track of the account balances of e in a Merkle tree [44];
- 2) at the end of each epoch, the block proposer creates a state block $sb(a, e)$, which stores the hash of the Merkle tree's root in the block's header and the balances in the block's body;
- 3) validators verify and run consensus on $sb(a, e)$;
- 4) if consensus is reached, the block proposer will store $sb(a, e)$ in the shard's ledger, making it the genesis block for epoch $e + 1$;
- 5) at the end of epoch $e + 1$, nodes will drop the body of $sb(a, e)$ and all blocks preceding $sb(a, e)$.

Using this mechanism, the bootstrapping of the new nodes should be very efficient. Actually, they start only from the last valid state block and compute only the following blocks instead of its full history.

X Security Evaluation

1 Randomness source

Elrond makes use of random numbers in its operation e.g. for the random sampling of block proposer and validators into consensus groups and the shuffling of nodes between shards at the end of an epoch. Because these features contribute to Elrond's security guarantees, it is therefore important to make use of random numbers that are provably unbiased and unpredictable. In addition to these properties, the generation

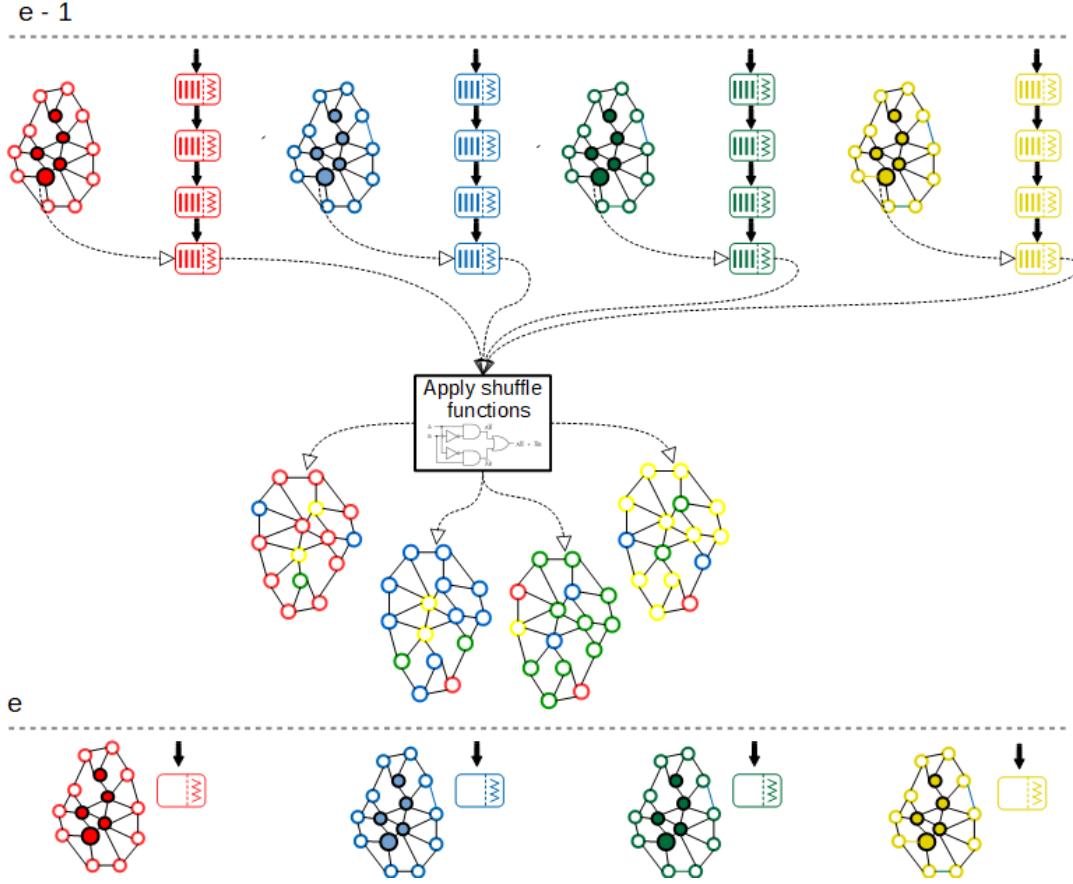


Fig. 10: Shuffling the nodes at the end of each epoch

of random numbers also needs to be efficient so that it can be used in a scalable and high throughput blockchain architecture.

These properties can be found in some asymmetric cryptography schemes, like the BLS signing scheme. One important property of BLS is that using the same private key to sign the same message always produces the same results. This is similar to what is achieved using ECDSA with deterministic key generation and is due to the scheme not using any random parameters:

$$sig = sk \cdot H(m) \quad (8)$$

where H is a hashing function that hashes to points on the used curve and sk is the private key.

2 Randomness creation in Elrond

One random number is created in every round, and added by the block proposer to every block in the blockchain. This ensures that the random numbers are unpredictable, as each random number is the signature of a different block proposer over the previous randomness source. The creation of random numbers is detailed below as part of one consensus round:

- 1) New consensus group is selected using the randomness source from the previous block header. Consensus group is formed by a block proposer and validators.
- 2) The block proposer signs the previous randomness source with BLS, adds the signature to the proposed

block header as new randomness source, then broadcasts this block to the consensus group.

- 3) Each member of the consensus group validates the randomness source as part of block validation, and sends their block signature to the block proposer.
- 4) Block proposer aggregates the validators block signatures and broadcasts the block with the aggregated block signature and the new randomness source to the whole shard.

The evolution of randomness source in each round can be seen as an unbiased and verifiable blockchain, where each new random number can be linked to and verified against the previous random number.

3 "K" block finality scheme

The signed block at round n is final, if and only if blocks $n + 1, n + 2, \dots, n + k$ are signed. Furthermore, a final block cannot be reverted. The metachain notarizes only final blocks to ensure that a fork in one shard does not affect other shards. Shards only take into consideration the final metachain blocks, in order to not be affected if the metachain forks. Finality and correctness is verified at block creation and at block validation as well. The chosen k parameter is 1 and this ensures forks of maximum 2 blocks length. The probability that a malicious super majority ($> \frac{2}{3} \cdot n + 1$) is selected in the shard for the

same round in the same consensus is 10^{-9} , even if 33% of the nodes from the shard are malicious. In that case they can propose a block and sign it - let's call it *block m*, but it will not be notarized by the metachain. The metachain notarizes *block m*, only if *block m+1* is built on top of it. In order to create *block m+1* the next consensus group has to agree with *block m*. Only a malicious group will agree with *block m*, so the next group must have a malicious super majority again. As the random seed for group selection cannot be tampered with, the probability of selecting one more malicious super majority group is 10^{-9} ($5.38 \cdot 10^{-10}$, to be exact). The probability of signing two consecutive malicious blocks equals with selecting two subgroups with at least $(\frac{2}{3} \cdot n + 1)$ members from the malicious group consequently. The probability for this is 10^{-18} . Furthermore, the consequently selected groups must be colluding, otherwise the blocks will not be signed.

4 Fisherman challenge

When one invalid block is proposed by a malicious majority, the shard state root is tampered with an invalid result (after including invalid changes to the state tree). By providing the combined merkle proof for a number of accounts, an honest node could raise a challenge with a proof. The honest nodes will provide the block of transactions, the previous reduced merkle tree with all affected accounts before applying the challenged block and the smart contract states, thus demonstrating the invalid transaction / state. If a challenge with the proof is not provided in the bounded time frame, the block is considered valid. The cost of one invalid challenge is the entire stake of the node which raised the challenge.

The metachain detects the inconsistency, either an invalid transaction, or an invalid state root, through the presented challenges and proofs. This can be traced and the consensus group can be slashed. At the same time the challenger can be rewarded with part of the slashed amount. Another problem is when a malicious group hides the invalid block from other nodes - non-malicious ones. However, by making it mandatory for the current consensus to propagate the produced block to the sibling shard and to the observer nodes, the data cannot be hidden anymore. The communication overhead is further reduced by sending only the intrashard miniblock to the sibling shard. The cross shard miniblocks are always sent on different topics accessible by interested nodes. In the end, challenges can be raised by multiple honest nodes. Another security protection is given by the setup of P2P topics. The communication from one shard toward the metachain is done through a defined set of topics / channels, which can be listened to by any honest validator - the metachain will not accept any other messages from other channels. This solution introduces some delay in the metachain only in case of challenges, which are very low in number and highly improbable since if detected (high probability of being detected) the nodes risk their entire stake.

5 Shard reorganization

After each epoch, less than $\frac{1}{3} \cdot n$ of the nodes from each shard are redistributed uniformly and non-deterministically

across the other shards, to prevent collusion. This method adds bootstrapping overhead for the nodes that were redistributed, but doesn't affect liveness as shuffled nodes do not participate in the consensus in the epoch they have been redistributed. The pruning mechanism will decrease this time to a feasible amount, as explained in section IX.2.

6 Consensus group selection

After each round a new set of validators are selected using the random seed of the last committed block, current round and the eligible nodes list. In case of network desynchronization due to the delays in message propagation, the protocol has a recovery mechanism, and takes into consideration both the round r and the randomness seed from the last committed block in order to select new consensus groups every round. This avoids forking and allows synchronization on last block.

The small time window (round time) in which the validators group is known, minimizes the attack vectors.

7 Node rating

Beside stake, the eligible validator's rating influences the chances to be selected as part of the consensus group. If the block proposer is honest and its block gets committed to the blockchain, it will have its rating increased, otherwise, its rating will be decreased. This way, each possible validator is incentivized to be honest, run the most up-to-date client software version, increase its service availability and thus ensuring the network functions as designed.

8 Shard redundancy

The nodes that were distributed in sibling shards on the tree's lowest level (see section IV.4) keep track of each other's blockchain data and application state. By introducing the concept of shard redundancy, when the number of nodes in the network decreases, some of the sibling shards will need to be merged. The targeted nodes will instantly initiate the process of shard merging.

XI Understanding the real problems

1 Centralized vs Decentralized

Blockchain was initially instantiated as an alternative to the centralized financial system of systems [45]. Even if the freedom and anonymity of distributed architectures remains an undisputed advantage, the performance has to be analyzed at a global scale in a real-world environment.

The most relevant metric measuring performance is transactions per second (TPS), as seen in Table 2. A TPS comparison of traditional centralized systems with decentralized novel architectures that were validated as trusted and efficient on a large scale, reflects an objective yet unsettling reality [46], [47], [48], [49].

The scalability of blockchain architectures is a critical but still unsolved problem. Take, for instance, the example determining the data storage and bootstrapping implications of current blockchain architectures suddenly functioning at Visa level throughput. By performing such exercises, the magnitude of multiple secondary problems becomes obvious (see Fig. 11).

Architecture	Type	Dispersion	TPS (average)	TPS (max limit)
VISA	Distributed virtualization	Centralized	3500	55000
Paypal	Distributed virtualization	Centralized	200	450
Ripple	Private Blockchain	Permissioned	1500	55000
NEO	Private Blockchain	Mixed	1000	10000
Ethereum	Public Blockchain	Decentralized	15	25
Bitcoin	Public Blockchain	Decentralized	2	7

TABLE 2: Centralized vs Decentralized TPS comparison

XII The blockchain performance paradigm

The process of designing distributed architectures on blockchain faces several challenges, perhaps one of the most challenging being the struggle to maintain operability under contextual pressure conditions. The main components that determine the performance pressure are:

- complexity
- system size
- transaction volume

Complexity

The first element that limits the system performance, is the consensus protocol. A more complicated protocol determines a bigger hotspot. In PoW consensus architectures a big performance penalty is induced by the mining complexity that aims to keep the system decentralized and ASIC resilient [50]. To overrun this problem PoS makes a trade-off, simplifies the network management by concentrating the computing power to a subset of the network, but yields more complexity on the control mechanism.

System size

Expanding the number of nodes in existing validated architectures forces a serious performance degradation and induces a higher computational price that must be paid. Sharding seems to be a good approach, but the shard size plays a major role. Smaller shards are agile but more likely to be affected by malicious groups, bigger shards are safer, but their reconfiguration affects the system liveness.

Transaction volume

With a higher relevance compared to the others, the last item on the list represents the transaction processing performance. In order to correctly measure the impact of this criteria, this must be analyzed considering the following two standpoints:

- C1 transaction throughput - how many transactions a system can process per time unit, known as TPS, an output of a system [51];
- C2 transaction finality - how fast one particular transaction is processed, referring to the interval between its launch and its finalization - an input to output path.

C1. Transaction throughput in single chain architectures is very low and can be increased by using workarounds such as sidechain [52]. In a sharded architecture like ours, the transaction throughput is influenced by the number of shards, the computing capabilities of the validators/block proposers and the messaging infrastructure [8]. In general, as displayed in Fig. 13, this goes well to the public, but despite the importance of the metric, it provides only a fragmented view.

C2. Transaction finality - A more delicate aspect that emphasizes that even if the system may have a throughput of 1000 TPS, it may take a while to process a particular transaction. Beside the computing capabilities of the validators/block proposers and the messaging infrastructure, the transaction finality is mainly affected by the dispatching algorithm (when the decision is made) and the routing protocol (where should the transaction be executed). Most of the existing state of the art architectures refuse to mention this aspect but from a user standpoint this is extremely important. This is displayed in

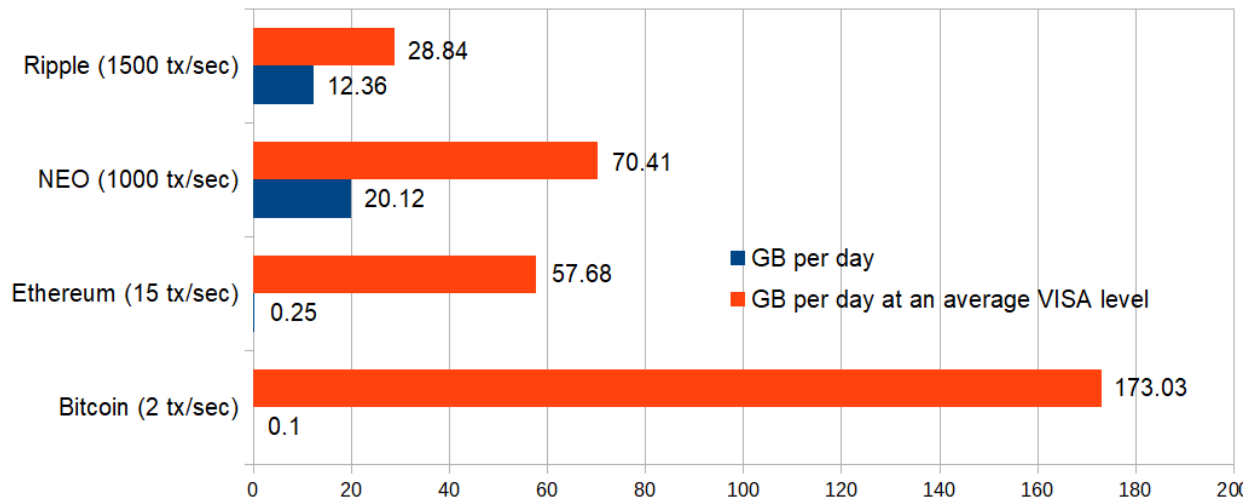
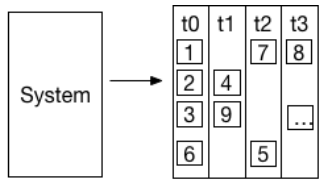


Fig. 11: Storage Estimation - Validated distributed architectures working at an average of VISA TPS



Processed transaction per time unit
Fig. 13: Transaction throughput

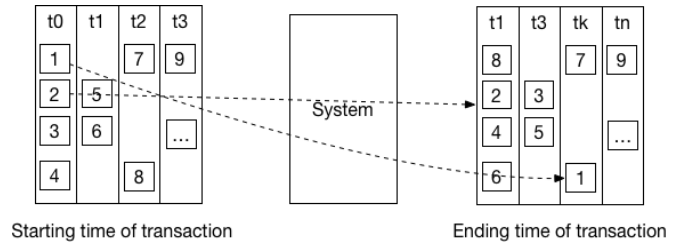


Fig. 14: Transaction finality

Fig. 14, where the total time required to execute a certain transaction from start to end is considered.

In Elrond, the dispatching mechanism (detailed in section V) allows an improved time to finality by routing the transactions directly to the right shard, mitigating the overall delays.

XIII Conclusion

1 Performance

Performance tests and simulations, presented in Fig. 12, reflect the efficiency of the solution as a highly scalable distributed ledger. As more and more nodes join the network our sharding approach shows a linearly increasing throughput. The chosen consensus model involves multiple communication rounds, thus the result is highly influenced by the network quality (speed, latency, availability). Simulations using our testnet using worldwide network speed averages, at its maximum theoretical limit, suggest Elrond exceeds the average VISA level with just 2 shards, and approaches peak VISA level with 16 shards.

2 Ongoing and future research

Our team is constantly re-evaluating and improving Elrond's design, in an effort to make this one of the most compelling public blockchain architectures; solving scalability via adaptive state sharding, while maintaining security and high energy efficiency through a secure Proof of Stake consensus mechanism. Some of our next directions of improvement include:

- 1) **Reinforcement learning:** we aim to increase the efficiency of the sharding process by allocating the frequently trading clients in the same shard to reduce the overall cost;
- 2) **AI supervision:** create an AI supervisor that detects malicious behavioral patterns; it is still uncertain how this feature can be integrated in the protocol without disrupting the decentralization;
- 3) **Reliability as a consensus factor:** the existing protocol weighs between stake and rating but we plan to add reliability, as a metric that should be computed in a distributed manner after applying a consensus protocol on previously submitted blocks from the very recent

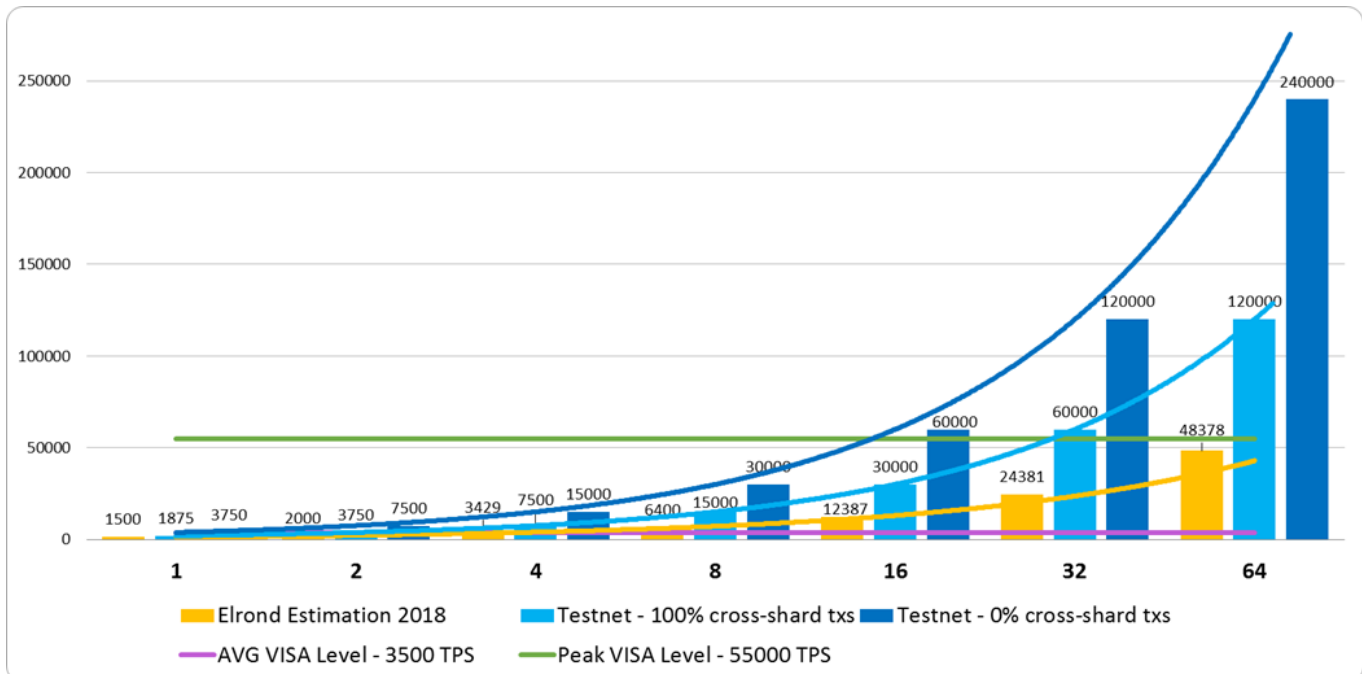


Fig. 12: Network throughput measured in transactions per seconds with a global network speed of 8 MB/s

history;

- 4) **Cross-chain interoperability:** implements and contribute to standards like those initiated by the Decentralized Identity Foundation [53] or the Blockchain Interoperability Alliance [54];
- 5) **Privacy preserving transactions:** use Zero-Knowledge Succinct Non-Interactive Argument of Knowledge [55] to protect the identity of the participants and offer auditing capabilities while preserving the privacy.

3 Overall Conclusions

Elrond is the first highly scalable public blockchain that uses the newly proposed Secure Proof of Stake algorithm in a genuine state-sharded architecture to achieve VISA level throughput and confirmation times of seconds. Elrond's novel approach on adaptive state sharding improves on Omniledger's proposal increasing security and throughput, while the built-in automatic transaction routing and state redundancy mechanisms considerably reduce latencies. By using a shard pruning technique the bootstrapping and storage costs are also considerably reduced compared to other approaches. The newly introduced Secure Proof of Stake consensus algorithm ensures distributed fairness and improves on Algorand's idea of random selection, reducing the time needed for the random selection of the consensus group from 12 seconds to 100 ms. Our method of combining state sharding and the very efficient Secure Proof of Stake consensus algorithm has shown promising results in our initial estimations, validated by our latest testnet results.

References

- [1] G. Hileman and M. Rauchs, "2017 Global Cryptocurrency Benchmarking Study," Social Science Research Network, Rochester, NY, SSRN Scholarly Paper ID 2965436, Apr. 2017. [Online]. Available: <https://papers.ssrn.com/abstract=2965436>
- [2] "The Ethereum Wiki - Sharding FAQ," 2018, original-date: 2014-02-14T23:05:17Z. [Online]. Available: <https://github.com/ethereum/wiki/wiki/Sharding-FAQ>
- [3] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling Byzantine Agreements for Cryptocurrencies," in *Proceedings of the 26th Symposium on Operating Systems Principles*, ser. SOSP '17. New York, NY, USA: ACM, 2017, pp. 51–68. [Online]. Available: <http://doi.acm.org/10.1145/3132747.3132757>
- [4] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Advances in Cryptology – ASIACRYPT '01, LNCS*. Springer, 2001, pp. 514–532.
- [5] D. Boneh, M. Drijvers, and G. Neven, "Compact multi-signatures for smaller blockchains," in *Advances in Cryptology – ASIACRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 11273. Springer, 2018, pp. 435–464.
- [6] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2013. [Online]. Available: <https://www.ethereum.org/pdfs/EthereumWhitePaper.pdf>
- [7] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding," Tech. Rep. 406, 2017. [Online]. Available: <https://eprint.iacr.org/2017/406>
- [8] "The ZILLIQA Technical Whitepaper," 2017. [Online]. Available: <https://docs.zilliqa.com/whitepaper.pdf>
- [9] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, "Chainspace: A Sharded Smart Contracts Platform," *arXiv:1708.03778 [cs]*, Aug. 2017, arXiv: 1708.03778. [Online]. Available: <http://arxiv.org/abs/1708.03778>
- [10] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," 2017. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [11] "Solidity — Solidity 0.4.21 documentation." [Online]. Available: <https://solidity.readthedocs.io/en/v0.4.21/>
- [12] "web3j," 2018. [Online]. Available: <https://github.com/web3j>
- [13] "Casper," 2018. [Online]. Available: <http://ethresear.ch/c/casper>
- [14] "The State of Ethereum Scaling, March 2018 – Highlights from EthCC on Plasma Cash, Minimum Viable Plasma, and More... – Medium," 2018. [Online]. Available: <https://medium.com/loom-network/the-state-of-ethereum-scaling-march-2018-74ac08198a36>
- [15] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, ser. OSDI '99. Berkeley, CA, USA: USENIX Association, 1999, pp. 173–186. [Online]. Available: <http://dl.acm.org/citation.cfm?id=296806.296824>
- [16] Y. Jia, "Op Ed: The Many Faces of Sharding for Blockchain Scalability," 2018. [Online]. Available: <https://bitcoinmagazine.com/articles/op-ed-many-faces-sharding-blockchain-scalability/>
- [17] "Using Merkle tree to shard block validation | Deadalnix's den," 2016. [Online]. Available: <https://www.deadnix.me/2016/11/06/using-merkle-tree-to-shard-block-validation/>
- [18] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," p. 9, 2008.
- [19] "Why we are building Cardano - Introduction." [Online]. Available: <https://whycardano.com/>
- [20] "Constellation - a blockchain microservice operating system - White Paper," 2017, original-date: 2018-01-05T20:42:05Z. [Online]. Available: <https://github.com/Constellation-Labs/Whitepaper>
- [21] "Bitshares - Delegated Proof-of-Stake Consensus," 2014. [Online]. Available: <https://bitshares.org/technology/delegated-proof-of-stake-consensus/>
- [22] dantheman, "DPOS Consensus Algorithm - The Missing White Paper," May 2017. [Online]. Available: <https://steemit.com/dpos/@dantheman/dpos-consensus-algorithm-this-missing-white-paper>
- [23] "EOS.IO Technical White Paper v2," 2018, original-date: 2017-06-06T07:55:17Z. [Online]. Available: <https://github.com/EOSIO/Documentation/blob/master/TechnicalWhitePaper.md>
- [24] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Berlin Heidelberg: Springer-Verlag, 2010. [Online]. Available: <http://www.springer.com/gp/book/9783642041006>
- [25] C. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, pp. 161–174, Jan. 1991.
- [26] K. Michaelis, C. Meyer, and J. Schwenk, "Randomly Failed! The State of Randomness in Current Java Implementations," in *Topics in Cryptology – CT-RSA 2013*, ser. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, Feb. 2013, pp. 129–144. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-36095-4_9
- [27] D. J. Bernstein, P. Birkner, M. Joye, T. Lange, and C. Peters, "Twisted Edwards Curves," in *Progress in Cryptology – AFRICACRYPT 2008*, ser. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, Jun. 2008, pp. 389–405. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-68164-9_26
- [28] A. Poelstra, "Schnorr Signatures are Non-Malleable in the Random Oracle Model," 2014. [Online]. Available: <https://download.wpsoftware.net/bitcoin/wizardry/schnorr-mall.pdf>
- [29] C. Decker and R. Wattenhofer, "Bitcoin Transaction Malleability and MtGox," *arXiv:1403.6676 [cs]*, vol. 8713, pp. 313–326, 2014, arXiv: 1403.6676. [Online]. Available: <http://arxiv.org/abs/1403.6676>
- [30] G. Maxwell, A. Poelstra, Y. Seurin, and P. Wuille, "Simple Schnorr Multi-Signatures with Applications to Bitcoin," Tech. Rep. 068, 2018. [Online]. Available: <https://eprint.iacr.org/2018/068>
- [31] Y. Seurin, "On the Exact Security of Schnorr-Type Signatures in the Random Oracle Model," in *Advances in Cryptology – EUROCRYPT 2012*, ser. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, Apr. 2012, pp. 554–571. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-29011-4_33
- [32] K. Itakura and K. Nakamura, "A public-key cryptosystem suitable for digital multisignatures," 1983.
- [33] S. Micali, K. Ohta, and L. Reyzin, "Accountable-subgroup Multisignatures: Extended Abstract," in *Proceedings of the 8th ACM Conference on Computer and Communications Security*, ser. CCS '01. New York, NY, USA: ACM, 2001, pp. 245–254. [Online]. Available: <http://doi.acm.org/10.1145/501983.502017>
- [34] T. Ristenpart and S. Yilek, "The Power of Proofs-of-Possession: Securing Multiparty Signatures against Rogue-Key Attacks," in *Advances in Cryptology - EUROCRYPT 2007*, ser. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, May 2007, pp.

- 228–245. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-72540-4_13
- [35] M. Bellare and G. Neven, “Multi-signatures in the Plain public-Key Model and a General Forking Lemma,” in *Proceedings of the 13th ACM Conference on Computer and Communications Security*, ser. CCS ’06. New York, NY, USA: ACM, 2006, pp. 390–399. [Online]. Available: <http://doi.acm.org/10.1145/1180405.1180453>
- [36] D.-P. Le, A. Bonnet, and A. Gabillon, “Multisignatures as Secure as the Diffie-Hellman Problem in the Plain Public-Key Model,” in *Pairing-Based Cryptography – Pairing 2009*, ser. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, Aug. 2009, pp. 35–51. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-03298-1_3
- [37] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen, “Adding Concurrency to Smart Contracts,” in *Proceedings of the ACM Symposium on Principles of Distributed Computing*, ser. PODC ’17. New York, NY, USA: ACM, 2017, pp. 303–312. [Online]. Available: <http://doi.acm.org/10.1145/3087801.3087835>
- [38] J. Kwon and E. Buchman, “Cosmos Network - Internet of Blockchains,” 2017. [Online]. Available: <https://cosmos.network/whitepaper>
- [39] G. Roşu and T. F. Şerbănuţă, “An overview of the k semantic framework,” *The Journal of Logic and Algebraic Programming*, vol. 79, no. 6, pp. 397–434, 2010.
- [40] T. Kasampalis, D. Guth, B. Moore, T. Serbanuta, V. Serbanuta, D. Filaratti, G. Rosu, and R. Johnson, “Iele: An intermediate-level blockchain language designed and implemented using formal semantics,” Tech. Rep., 2018.
- [41] E. Hildenbrandt, M. Saxena, X. Zhu, N. Rodrigues, P. Daian, D. Guth, and G. Rosu, “Kevm: A complete semantics of the ethereum virtual machine,” Tech. Rep., 2017.
- [42] “How Formal Verification of Smart Contracts Works | RV Blog,” [Online]. Available: <https://runtimeverification.com/blog/how-formal-verification-of-smart-contracts-works/>
- [43] “Cross-shard contract yanking,” [Online]. Available: <https://ethresear.ch/t/cross-shard-contract-yanking/1450>
- [44] R. C. Merkle, “A Certified Digital Signature,” in *Advances in Cryptology — CRYPTO’ 89 Proceedings*, ser. Lecture Notes in Computer Science. Springer, New York, NY, Aug. 1989, pp. 218–238. [Online]. Available: https://link.springer.com/chapter/10.1007/0-387-34805-0_21
- [45] A. Veysov and M. Stolbov, “Financial System Classification: From Conventional Dichotomy to a More Modern View,” Social Science Research Network, Rochester, NY, SSRN Scholarly Paper ID 2114842, Jul. 2012. [Online]. Available: <https://papers.ssrn.com/abstract=2114842>
- [46] “XRP - The Digital Asset for Payments,” [Online]. Available: <https://ripple.com/xrp/>
- [47] “Visa - Annual Report 2017,” 2018. [Online]. Available: https://s1.q4cdn.com/050606653/files/doc_financials/annual/2017/Visa-2017-Annual-Report.pdf
- [48] “PayPal Reports Fourth Quarter and Full Year 2017 Results (NASDAQ:PYPL),” 2018. [Online]. Available: <https://investor.paypal-corp.com/releasedetail.cfm?releaseid=1055924>
- [49] M. Schwarz, “Crypto Transaction Speeds 2018 - All the Major Cryptocurrencies,” 2018. [Online]. Available: <https://www.abitgreedy.com/transaction-speed/>
- [50] “The Ethereum Wiki - Mining,” 2018, original-date: 2014-02-14T23:05:17Z. [Online]. Available: <https://github.com/ethereum/wiki/wiki/Mininghttps://github.com/ethereum/wiki>
- [51] “Transaction throughput,” [Online]. Available: https://docs.oracle.com/cd/E17276_01/html/programmer_reference/transapp_throughput.html
- [52] W. Martino, M. Quaintance, and S. Popejoy, “Chainweb: A Proof-of-Work Parallel-Chain Architecture for Massive Throughput,” 2018. [Online]. Available: <http://kadena.io/docs/chainweb-v15.pdf>
- [53] “DIF - Decentralized Identity Foundation,” [Online]. Available: <http://identity.foundation/>
- [54] H. I. World, “Blockchain Interoperability Alliance: ICON x Aion x Wanchain,” Dec. 2017. [Online]. Available: <https://medium.com/helloiconworld/blockchain-interoperability-alliance-icon-x-aion-x-wanchain-8aeaafb3ebdd>
- [55] S. Goldwasser, S. Micali, and C. Rackoff, “The Knowledge Complexity of Interactive Proof-systems,” in *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, ser. STOC ’85. New York, NY, USA: ACM, 1985, pp. 291–304. [Online]. Available: <http://doi.acm.org/10.1145/22145.22178>